COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S4
DATE PERFORMED:	10/07/26	**DATE SUBMITTED:**	10/09/26
NAME:	Garcia, Dennis James T. Hare, Andrei Angelo D.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

HANDS-ON ACTIVITY BASIC ALGORITHM ANALYSIS

1. PROCEDURE OUTPUT

ILO A: Measure the runtime of algorithms using theoretical analysis

Assess if two arrays have any common values.

```
bool diff(int *x, int *y){
   for(int i = 0; i < y.length; i++){
      if(search(x, y[i])!= - 1){
         return false;
      }
   }
   return true;
}
```

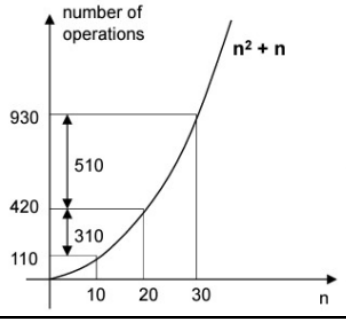
Performing worst case analysis:

- Let m = the length of array x.
- Let n = the length of array y.
- The loop in diff repeats n times.
- Each call to search requires m comparisons.

Given the above information, the total number of comparisons in the worst case is: _____

- Assume that m = n. (The arrays are the same size)

Provide an analysis of the graph for the worst case of the algorithm.



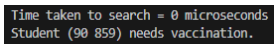
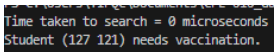
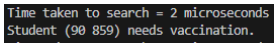
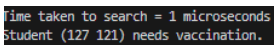
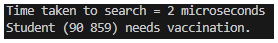
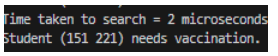
n (size of array)	number of operations
10	110
20	420
30	930

Analysis (by pair)

We would assume that the number of loop runs increases alongside with the number of operations; thus affecting execution time. Judging by the graph, it would take more time to process the algorithm if the algorithm has a lot of loops and the number of operations is too many. The graph is gradually leading upwards and increasing in execution time due to a large number of operations and loops.

ILO B: Analyze the results of runtime performance by comparing experimental and theoretical analysis

Fill out the table below and include in section 6 table 11-1.

Input Size (N)	Execution Speed	Screenshot	Observation(s)
1000	Hare: 0 microseconds Garcia: 0 microseconds	Hare:  Garcia:  	Hare: The program runs quickly because the input size was small and the computer processed it very fast Garcia: Based on the output, The program runs fast because the input size was small and the computer processed it quickly
10000	Hare: 2 microseconds Garcia: 1 microseconds	Hare:  Garcia:  	Hare: The program took 2 microseconds to process because the input size increases causing to Garcia: Judging by the screenshots it would seem that my computer processed the code by a microsecond faster than my partner's computer. Using an input size of ten-thousand.
100000	Hare: 2 microseconds Garcia: 1 microseconds	Hare:  Garcia:  	Hare: The program was still fast because the execution time was very low even though the input size was larger compared to the other two. Garcia: According to the screenshots, me and my partner's computers have the same processing time of two microseconds,

			despite having the same code the only difference is the program's input size; We both had the same execution time on different devices

Provide your own analysis of the outcome listed in this table.

Hare

- The results may vary because of the input size and some factors like computer's processing speed, internet access, etc. but the overall output of the three was still fast.

Garcia

- The outcome of the execution time may differ due to hardware differences. The range of the seconds we can output is only between 0 to 3 microseconds. Even if there is variance in our hardware, It seems that this is the only range we can seem to generate with the *generate_random_Student* function from the program/code that we are using to determine the difference in execution time of algorithms.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO A: Measure the runtime of algorithms using theoretical analysis & ILO B: Analyze the results of runtime performance by comparing experimental and theoretical analysis

For each of the problems below, provide the following:

- Theoretical Analysis
- Experimental Analysis
- Analysis and Comparison

Problem 1: Consider a program that returns true if the elements in an array are unique.

```
Algorithm Unique(A)
  for i = 0 to n-1 do
    for j = i+1 to n-1 do
      if A[i] equals to A[j] then
        return false
  return true
```

<table>
<tr><td>Theoretical Analysis</td><td>Hare
Based on the Algorithm it uses two loops, so the number of comparisons increases as the array size increases.

Garcia
Looking at the algorithm it uses a nested loop, therefore the amount of runs a loop takes increases with the amount of comparisons, specifically if the array size is large. so there might be a delay.</td></tr>
<tr><td></td><td>Time complexities:

<table><tr><td>Worst Time Complexity</td><td>$O(n^2)$, due to the nested loop</td></tr><tr><td>Best Time Complexity</td><td>O(n), only one loop is involved</td></tr></table></td></tr>
<tr><td>Experimental Analysis</td><td>Hare
For me the program may take longer with a larger array because there are more elements to compare.

Garcia
The program may produce around a 1000 microsecond execution time difference that was tracked using std::chrono</td></tr>
</table>

Analysis and Comparison	Hare Based on the result, it shows that larger inputs require more comparisons because the program needs to check more elements. Garcia Based on the theoretical and experimental analysis, I conclude larger values can affect the runtime of the program due to the amount of runs a loop takes and the amount of comparison operators that the program must do depending on the array size.

Problem 2: Consider another program that raises a number x to an arbitrary nonnegative integer, n.

```
Algorithm rpower(int x, int n):
1 if n == 0 return 1
2 else return x*rpower(x, n-1)

Algorithm brpower(int x, int n):
1 if n == 0 return 1
2 if n is odd then
3   y = brpower(x, (n-1)/2)
4   return x*y*y
5 if n is even then
6   y = brpower(x, n/2)
7   return y*y
```

Theoretical Analysis	Hare rpower makes more recursive calls because it decreases the exponent by 1, while brpower reduces it by half. Garcia The first algorithm decrements n by one and x being the base number you want to multiply n with. If the n value is too high, you'll create a long chain of recursive calls making the execution time long. The brpower algorithm does the same thing as the rpower algorithm but it divides n by half; thus less computations.

<table>
<tr>
<td></td>
<td>Time complexities:

Rpower

<table>
<tr><td>Worst Time Complexity</td><td>O(n^2), due to the constant recursive calls from rpower algorithm and when every element in the array is unique causing it to check every element and its pair.</td></tr>
<tr><td>Best Time Complexity</td><td>O(n), if we are using a massive exponent value for n then it steps down n by mins one.</td></tr>
</table>

Brpower

<table>
<tr><td>Worst Time Complexity and Best Time Complexity</td><td>O(log n), it divides the n by 2 each time until the n reaches 0. This helps by reducing the amount of recursive calls by cutting the n by half therefore lessening execution time and decreasing the amount of times you compare elements.</td></tr>
</table>
</td>
</tr>
<tr>
<td>Experimental Analysis</td>
<td>Hare
rpower may take longer for larger values of n because it makes more recursive calls.

Garcia
Rpower took way longer than brpower due to its algorithm only decrementing n by one instead of half. The execution time was around 7000 microseconds for the rpower algorithm while only 6700 microseconds for the br power that was tracked using std::chrono .</td>
</tr>
<tr>
<td>Analysis and Comparison</td>
<td>Hare
I believe brpower is faster for larger inputs because it reduces the problem faster than rpower.

Garcia
In conclusion, both the theoretical analysis and experimental analysis were on point. Due to the rpower algorithm only decrementing by one, the first algorithm took too many recursive calls while the brpower algorithm took less recursive calls because it decrements n by half.</td>
</tr>
</table>

Conclusion:

Hare	
Summary of lessons learned	For me I learned that the runtime of the algorithm depends on the number of operations and the size of the input. I also learned that some algorithms are faster because they need fewer steps to finish.
Analysis of the procedure	The results show that the execution time can differ slightly between computers even when using the same code. I also noticed that the measured times stayed very low even when the input size increased.
Analysis of the supplementary activity	I learned that the Unique algorithm needs more comparisons when the array gets larger. I also learned that brpower can be faster than rpower because it reduces the exponent by half instead of decreasing it by one.
Concluding statement / Feedback: How well did you think you did in this activity? What are your areas for improvement?	I think we did well in this activity because we were able to understand and compare the algorithms but I still need more practice in analyzing runtime and understanding how the number of operations affects program performance.

Garcia	
Summary of lessons learned	I learned how to analyze the runtime of algorithms using theoretical and experimental analysis. I also learned that the number of operations, loops, and recursive calls can affect how long a program takes to execute.
Analysis of the procedure	Based on the graph and our experimental results, the number of operations should increase as the input size becomes larger. However, our measured execution times only showed a small difference between the input sizes. It shows that the actual execution time may vary depending on how the program measures its runtime and the computer being used.
Analysis of the supplementary activity	Based on the theoretical and experimental analysis I learned that the Unique algorithm uses nested loops which increase the number of comparisons as the array size grows. For the power algorithms, rpower took more recursive calls because it only decrements the exponent by one, while brpower reduces it by half that makes it more efficient.
Concluding statement / Feedback: How well did you think you did in this activity? What are your areas for improvement?	In conclusion, I think we were able to understand the basic concepts of algorithm analysis through this activity. By comparing the algorithms, I learned that the number of operations and recursive calls can affect program performance. However I still need to improve my understanding of runtime measurements and practice analyzing different algorithms to better understand their efficiency.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used:

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use:

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.